



UNIVERSIDADE FEDERAL DO TOCANTINS
CAMPUS UNIVERSITÁRIO DE PALMAS
CURSO DE CIÊNCIA DA COMPUTAÇÃO

Laboratório 1 – Minimização Lógica e Hazards

Pedro Lucas Moreira

Relatório – Linguagens de Descrição de Hardware – 2026/2 – Agosto de 2026

Professor(a) Orientador(a): Tiago Almeida

The contents of this report are the sole responsibility of the authors.
O conteúdo deste relatório é de única responsabilidade dos autores.

Pedro Lucas Moreira

Laboratório 1 – Minimização Lógica e Hazards

Relatório apresentado como parte das atividades da disciplina Linguagens de Descrição de Hardware – 2026/2, Curso de Ciência da Computação, Universidade Federal do Tocantins.

PALMAS / TO

2026

1 QUESTÃO 1 – MINIMIZAÇÃO LÓGICA

1.1 Expressão original

A função foi obtida a partir do mapa de Karnaugh de 5 variáveis (Figura 1):

$$F(A, B, C, D, E) = \sum m(0, 2, 5, 8, 13, 18, 21, 24, 29, 31)$$

1.2 Minimização por mapa de Karnaugh

A=0				
DE BC	00	01	11	10
00	1 0	0 1	0 3	1 2
01	0 4	1 5	0 7	0 6
11	0 12	1 13	0 15	0 14
10	1 8	0 9	0 11	0 10

A=1				
DE BC	00	01	11	10
00	0 16	0 17	0 19	1 18
01	0 20	1 21	0 23	0 22
11	0 28	1 29	1 31	0 30
10	1 24	0 25	0 27	0 26

Figura 1 – Mapa de Karnaugh de 5 variáveis (tabelas $A = 0$ e $A = 1$).

Agrupando os 1's do mapa, obtém-se a expressão minimizada:

$$F = ABCE + \overline{B}\overline{C}D\overline{E} + B\overline{C}\overline{D}\overline{E} + C\overline{D}E + \overline{A}\overline{B}\overline{C}\overline{E}$$

Em notação de linha (usada no restante do documento e no código-fonte):

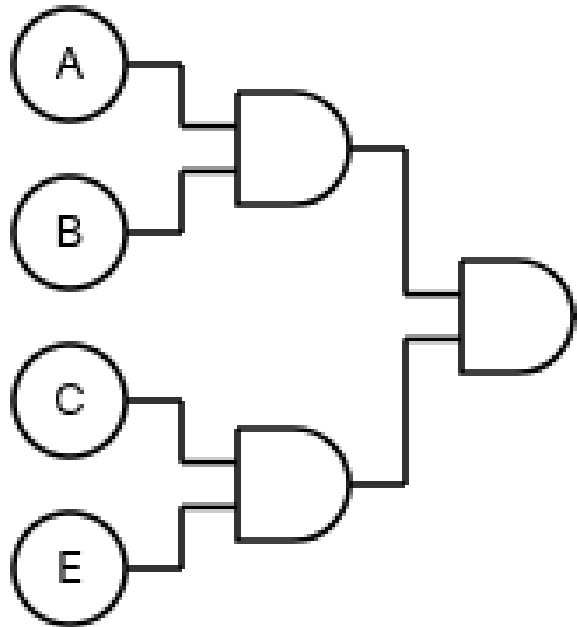
$$F = ABCE + B'C'DE' + BC'D'E' + CD'E + A'B'C'E'$$

1.3 Tabela verdade

Tabela 1 – Tabela verdade completa (32 combinações).

A	B	C	D	E	F
0	0	0	0	0	1
0	0	0	0	1	0
0	0	0	1	0	1
0	0	0	1	1	0
0	0	1	0	0	0
0	0	1	0	1	1
0	0	1	1	0	0
0	0	1	1	1	0
0	1	0	0	0	1
0	1	0	0	1	0
0	1	0	1	0	0
0	1	0	1	1	0
0	1	1	0	0	0
0	1	1	0	1	1
0	1	1	1	0	0
0	1	1	1	1	0
1	0	0	0	0	0
1	0	0	0	1	0
1	0	0	1	0	1
1	0	0	1	1	0
1	0	1	0	0	0
1	0	1	0	1	1
1	0	1	1	0	0
1	0	1	1	1	0
1	1	0	0	0	1
1	1	0	0	1	0
1	1	0	1	0	0
1	1	0	1	1	0
1	1	1	0	0	0
1	1	1	0	1	1
1	1	1	1	0	0
1	1	1	1	1	1

1.4 Diagrama lógico



1.5 Implementação VHDL estrutural (sem minimização)

```
1 library ieee;
2 use ieee.std_logic_1164.all;
3
4 entity and_gate is
5     port (
6         A    : in  std_logic;
7         B    : in  std_logic;
8         C    : in  std_logic;
9         D    : in  std_logic;
10        E    : in  std_logic;
11
12        F    : out std_logic
13    );
14 end entity and_gate;
15
16 architecture rtl of and_gate is
17 begin
18     F <= (not A and not B and not C and not D and not E)
```

```

19      or (not A and not B and not C and D and not E)
20      or (not A and not B and C and not D and E)
21      or (not A and B and not C and not D and not E)
22      or (not A and B and C and not D and E)
23      or (A and not B and not C and D and not E)
24      or (A and not B and C and not D and E)
25      or (A and B and not C and not D and not E)
26      or (A and B and C and not D and E)
27      or (A and B and C and D and E);
28 end architecture rtl;

```

Listing 1.1 – Circuito implementado como OR dos mintermos (e.vhd).

1.6 Simulação no Quartus

```

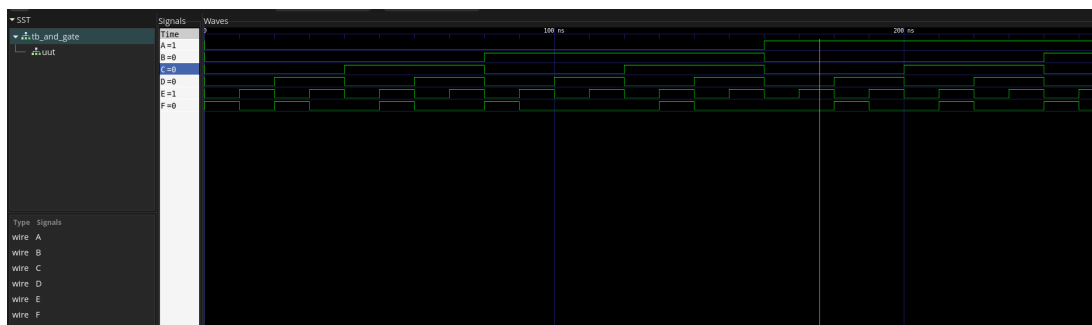
1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.numeric_std.all;
4
5  entity tb_and_gate is
6  end entity tb_and_gate;
7
8  architecture sim of tb_and_gate is
9      signal A, B, C, D, E, F : std_logic;
10 begin
11     uut: entity work.and_gate
12         port map (
13             A => A,
14             B => B,
15             C => C,
16             D => D,
17             E => E,
18             F => F
19         );
20
21     stim: process
22         variable v : std_logic_vector(4 downto 0);
23     begin
24         for i in 0 to 31 loop

```

```

25         v := std_logic_vector(to_unsigned(i, 5));
26         A <= v(4);
27         B <= v(3);
28         C <= v(2);
29         D <= v(1);
30         E <= v(0);
31         wait for 10 ns;
32     end loop;
33     wait;
34 end process;
35 end architecture sim;

```



1.7 Teste na DE1

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity and_gate is
5      port (
6          SW0    : in  std_logic;
7          SW1    : in  std_logic;
8          SW2    : in  std_logic;
9          SW3    : in  std_logic;
10         SW4    : in  std_logic;
11
12         LEDR1   : out std_logic
13     );
14
15
16 end entity and_gate;
17

```



```
18 architecture rtl of and_gate is
19 begin
20     LEDR1 <= (not SW0 and not SW1 and not SW2 and not SW3 and not SW4)
21         or (not SW0 and not SW1 and not SW2 and SW3 and not SW4)
22         or (not SW0 and not SW1 and SW2 and not SW3 and SW4)
23         or (not SW0 and SW1 and not SW2 and not SW3 and not SW4)
24         or (SW0 and not SW1 and not SW2 and SW3 and not SW4)
25         or (SW0 and not SW1 and SW2 and not SW3 and SW4)
26         or (SW0 and SW1 and not SW2 and not SW3 and not SW4)
27         or (SW0 and SW1 and SW2 and not SW3 and SW4)
28         or (SW0 and SW1 and SW2 and SW3 and SW4);
29
30 end architecture rtl;
```

2 QUESTÃO 2 – HAZARDS

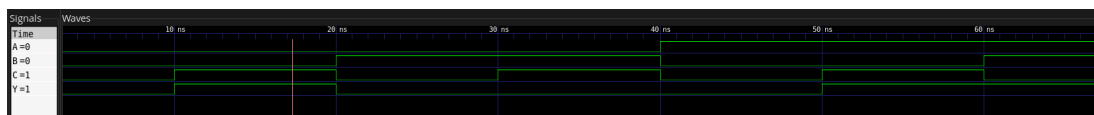
2.1 Implementação na DE1

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity and_gate is
5      port (
6          A    : in  std_logic;
7          B    : in  std_logic;
8          C    : in  std_logic;
9
10         Y    : out std_logic
11     );
12 end entity and_gate;
13
14 architecture rtl of and_gate is
15 begin
16
17     Y <= (A and B) or (not B and C);
18
19 end architecture rtl;

```

2.2 Simulação funcional



2.3 Simulação de timing

2.4 Explicação do hazard

O circuito implementa $Y = A \cdot B + \overline{B} \cdot C$. O sinal B aparece nos dois termos da soma, direto em um, invertido no outro, e cada caminho passa por um número diferente

de portas até chegar ao OR final (um deles atravessa um inversor extra). Isso gera um atraso de propagação desigual entre os dois termos.

A correção padrão é adicionar o termo de consenso redundante à expressão:

$$Y = A \cdot B + \overline{B} \cdot C + A \cdot C$$

Esse termo mantém a saída em 1 durante a transição de B sempre que $A = C = 1$, cobrindo a lacuna sem alterar a tabela-verdade do circuito.